

OpenWebText 句段预处理：设计动机、算法与文献背景

bdelf 数据集文档

2026 年 8 月 22 日

摘要

本文档说明 bdelf 仓库中 OpenWebText (OWT) 从「跨文档流式定长切块」迁移到「按文档句段切分 + 可选长度分桶」的动机与算法细节。工程配置、路径与构建命令见同目录上级 README.md。

1 背景与数据

OpenWebText 是从 Reddit 外链爬取的网页文本集合，规模与领域接近 GPT-2 训练所用的 WebText，常作为语言模型预训练与生成质量评测的基准语料 [1, 2]。在本仓库中，原始数据以 parquet 形式存放在 `cache/datasets/owt/`，逻辑上仅使用 Hugging Face 的 `train` split。

2 为何要改预处理

2.1 旧方案：跨文档流式切块 (`strategy: stream`)

历史配置（如 `default`、`default-old`）的流程为：

1. 对多篇文档分别做 BPE，将 token 序列**首尾相接**成一条长流；
2. 按固定长度 L （如 1024）切块，每块行首插入一个 BOS，尾部不足则按 `overflow_mode` 丢弃或填充。

该方案对自回归语言模型预训练简单有效 [2]，但存在明显结构问题：

- **样本边界与语义单元不一致**：一个 chunk 可包含多篇短文的尾部与下一篇的开头，模型看到的「序列」并非单一网页或段落。
- **不利于序列级目标**：变长编解码（VAE、全文重建）、按文档评估、或需要文档级 holdout 的任务会被跨篇拼接污染。
- **与「在自然边界切分」的常见实践不一致**：许多工作在长文档上先在句/段边界切分，再定长或分桶 [3]。

因此新预处理冻结旧行为于 `*-old`，并引入 `owt-seg512 / owt-bucket(strategy: owt_segment)`，在**单篇文档内**完成切分，且显式加入 BOS/EOS。

2.2 新方案概览

两篇文档独立处理，token 流不跨文档拼接。对每篇文档：

1. GPT-2 BPE 编码 (`add_special_tokens=False`);
2. 按长度上限 d 与分隔符规则切成若干 *content* 段;
3. 每段包装为 `[BOS, content, EOS]`，总长 $\leq d$;
4. 过滤过短段（有效长度 ≤ 128 ），再 pad 到定长或分桶长度;
5. 在 `train/dev/test` 各 split 内 shuffle 后写入缓存。

3 分隔符与切点选择

3.1 分隔符集合

在原文字符上匹配分隔符，再映射到 token 边界（见第 4 节）。优先级（数字越小越优先）为：

P0 连续空行（段落）;

P1 单个换行;

P2 句末：省略号、中文/英文句末标点 `.!?.!?`，及可选闭合引号、括号与尾随换行;

P3 分号（默认关闭，避免过碎切分）。

在长度接近上限时，优先在语义更完整的边界切分，而非任意 token 位置硬切，这与神经机器翻译中按子词单元切分 rare word 的思路不同 [4]：此处关注的是文档结构而非词表稀疏性。

3.2 Lookback 硬切回退

设单次处理上限为 d ，内容上限 $c = d - 2$ （预留 BOS、EOS）。对当前段起点 s ，若剩余 token 数 $> c$ ，令 $t = s + c$ 。在区间 $(\max(s, t - \ell), t]$ 内选切点，其中 $\ell = \min(256, \lfloor d/8 \rfloor)$ （例如 $d = 512 \Rightarrow \ell = 64$ ， $d = 2048 \Rightarrow \ell = 256$ ）。

在该窗口内：

- 仅考虑分隔符结束位置对应的 token 边界;
- 先按 $P0 > P1 > P2$ 选档位；同档取最靠右（最接近 t ）的切点;
- 若无分隔符，在 t 处硬切，下一段从 t 继续（不跳到下一分隔符之后，避免丢 token）。

若整篇 content 长度 $\leq c$ ，则一篇一段。该设计在「尊重边界」与「严格长度上限」之间折中：硬切不可避免时仍保持流式扫描，不二次全文搜索。

4 词表、BOS/EOS 与 offset 映射

采用 GPT-2 byte-level BPE [2]。特殊符号使用仓库扩展的 `<|bos|>`、`<|eos|>`、`<|pad|>`，而非依赖 HF 默认特殊 token，以便与下游 FL-TokenLayout 一致。

为何必须用 Fast tokenizer 的 offset_mapping? GPT-2 对 CJK 等字符常拆成多个 byte token，多个 token 可共享同一字符区间。若在 token 串上 decode 再搜标点，会与原文对齐失败。因此在 UTF-8 原文上用正则找分隔符，再用 `offset_mapping` 将字符结束位置映射为「包含该字符的最后一个 token 之后」的下标。

这与 SentencePiece 等「先归一化再切分」的流程不同 [5]，但目标相同：切点落在人类可读的边界上。

5 两套产物：定长与分桶

5.1 owt-seg512 ($d = 512$ ，定长 pad)

- 每段包装后 pad 到 512，右侧 `<|pad|>`；
- `.len` 记录有效长度（含 BOS/EOS），供训练时 mask padding；
- 适用于固定 $L = 512$ 的训练图（如单形状 `torch.compile`）。

5.2 owt-bucket ($d = 2048$ ，长度分桶)

切分上限提高到 2048，再按有效长度映射到 pad 桶：

$(128, 256] \rightarrow 256$, $(256, 512] \rightarrow 512$, $(512, 1024] \rightarrow 1024$, $(1024, 2048] \rightarrow 2048$.

动机：

- OWT 文档长度分布跨度大，统一 pad 到最大长度浪费算力；
- T5 等模型在固定张量长度下通过 packing / 多长度训练兼顾效率与覆盖 [3]；
- 分桶使同一 batch 内序列等长，便于注意力实现；manifest 记录 `bucket_counts` 供后续采样。

更长序列的外推仍是开放问题（ALiBi、Longformer 等 [6, 7]）；本预处理将 $d = 2048$ 作为切分上限而非声称模型外推能力。

6 数据划分：train / dev / test

在 parquet 行号上做一次随机划分 (`holdout_seed=42`)：先 1024 行 $\rightarrow$ test，再 1024 行 $\rightarrow$ dev，其余 $\rightarrow$ train。与旧版 `eval_count=10000` 的 holdout 无继承关系，指纹亦不同。

设计意图：

- dev：调参与训练中验证（待训练脚本接线）；

- `test`: 仅离线评测, 避免反复调参污染;
- 行级 holdout 保证同一网页不会同时出现在 train 与 dev/test。

7 打乱与可复现性

每个 split 内全部 chunk 生成后, 用 `shuffle_seed` (与 split 名混合) 生成置换, 再写入 shard。预处理 shuffle 与训练时按 step 的 epoch shuffle **独立**: 前者固定磁盘样本顺序, 后者仍可在训练循环中随机化。

配置与数据集字段进入 SHA-256 指纹 (16 hex), 缓存目录名 `owt_{preprocess}_{hex}`。改 holdout、改 `d`、改 pad 策略均会生成新目录; 不对旧缓存做软链别名。

8 与 BERT 式句对切分的对比

BERT 等使用「下一句预测」时在句边界拼接 [8], 但预训练 chunk 仍常为固定 512。本方案更接近「每段是带明确起止的文档片段」, 并显式 EOS, 适合生成式目标与全文重建类模型, 而非仅句对分类。

9 局限与后续工作

- 分隔符基于英文网页标点与换行启发式, 对列表、代码块、表格等结构可能切分不理想;
- P3 分号默认关闭; 开启会增加切分粒度;
- 训练侧尚未实现变长分桶采样与 dev 验证接线;
- ELF 等 T5 词表仍使用 `elf-old` 流式预处理, 未迁移到 `owt_segment`。

10 小结

新 OWT 预处理的核⼼是: **文档内、边界感知、上限为 d 的句段切分**, 配合 BOS/EOS 与可选长度分桶, 并在行级三分数据上构建可复现缓存。旧流式方案通过 `*-old` 保持可续训, 两者通过配置名与指纹隔离, 避免 silent 行为变更。

References

- [1] Aaron Gokaslan, Vanya Cohen, et al. *OpenWebText Corpus*. <https://skylion007.github.io/OpenWebTextCorpus/>. Skylion007/openwebtext on Hugging Face.
- [2] Alec Radford et al. “Language Models are Unsupervised Multitask Learners”. In: (2019). OpenAI Technical Report.

- [3] Colin Raffel, Noam Shazeer, Adam Roberts, et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research*. Vol. 21. 140. 2020, pp. 1–67.
- [4] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of ACL* (2016).
- [5] Taku Kudo and John Richardson. “SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing”. In: *Proceedings of EMNLP: System Demonstrations*. 2018.
- [6] Ofir Press, Noah A. Smith, and Mike Lewis. “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation”. In: *arXiv preprint arXiv:2108.12409* (2021).
- [7] Iz Beltagy, Matthew E. Peters, and Arman Cohan. “Longformer: The Long-Document Transformer”. In: *arXiv preprint arXiv:2004.05150*. 2020.
- [8] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT*. 2019.